

Table of Contents

Driver de motor Maxon (`Maxon_Node.py`) — Guía de uso en ROS2

Guía práctica y orientada a **principiantes de ROS2** para usar el driver de motor Maxon del proyecto `INSIA_control`.

Código fuente: `INSIA_control/DriverNodes/Maxon_Node.py`

Índice

1. [Introducción](#)
 2. [Conceptos básicos de ROS2 \(mínimos para esta guía\)](#)
 3. [Cómo se nombran el nodo y los topics](#)
 4. [Topics que PUBLICA el driver \(salidas\)](#)
 5. [Topics a los que se SUSCRIBE el driver \(entradas\)](#)
 6. [Máquina de estados y flujo típico](#)
 7. [Parámetros del driver](#)
 8. [Valores usados en cada vehículo](#)
 9. [Ejemplos de uso](#)
 10. [Diagnóstico rápido](#)
-

1. Introducción

`Maxon_Node.py` es el **nodo driver** que controla un motor a través de una controladora Maxon (**MCD 60W** con perfil *EPOS*, o **EPOS4**). Se usa típicamente para mover el **volante** (dirección) y el **freno** de los distintos vehículos del proyecto (ASCOD, iMiEV, maqueta/GDELS...).

El driver **no habla directamente con el motor**: traduce órdenes sencillas de ROS2 («ponte en esta posición», «aplica este par», «actíivate») a tramas **CANopen** que envía por un topic. Un segundo nodo, el *gateway* CAN (`CAN_Node.py`), es quien pone/lee esas tramas en el bus físico (CAN sobre Ethernet).

Cada instancia del driver controla **un** motor. En un vehículo hay varias instancias (p.ej. una para el volante y otra para el freno), cada una con su propio nombre.

2. Conceptos básicos de ROS2 (mínimos para esta guía)

- **Nodo:** un programa que participa en el sistema ROS2. `Maxon_Node.py` es un nodo.
- **Topic:** un «canal» con nombre por el que viajan mensajes. Un nodo *publica* (escribe) o se *suscribe* (lee).
- **Mensaje:** la estructura de datos que viaja por un topic (tiene campos con tipos). Aquí se usan mensajes propios del proyecto, del paquete **insia_msg**.
- **Namespace (espacio de nombres):** un prefijo que se antepone a los nombres. En este proyecto es el identificador del vehículo (ascod, imiev, gdels...).

Comandos de terminal que usarás en esta guía:

```
ros2 topic list           # ver todos los topics activos
ros2 topic echo <topic>   # ver en vivo lo que se publica en un
topic
ros2 topic info <topic>   # ver el tipo de mensaje de un topic
ros2 interface show insia_msg/msg/EPOSConsigna # ver los campos de
un mensaje
ros2 topic pub <topic> <tipo> '<datos yaml>'    # publicar un mensaje
a mano
```

Consejo: para practicar sin motor conectado, puedes lanzar solo el driver y usar `ros2 topic echo/ros2 topic pub` para ver y simular la comunicación.

3. Cómo se nombran el nodo y los topics

El nombre del nodo **no** es fijo. Aunque en el código aparece MCD60, al arrancar mediante los `launch/*.py` se le asigna un nombre concreto con `name=` (por ejemplo `MCD60_Volante`, `EPOS4_Freno`, `MCD60_Freno`). Ese nombre de ejecución es el que el driver usa para construir sus topics.

Además, todos los topics llevan por delante el **namespace del vehículo**, que se lee de `$ROS_WS/vehicle.yaml` (campo `id_vehicle`).

Reglas de nombres (siendo `<ns>` el vehículo y `<Nodo>` el nombre de ejecución):

Tipo de topic	Patrón	Ejemplo (vehículo ascod, nodo MCD60_Volante)
Topics propios del motor	/<ns>/<Nodo>/<Topic>	/ascod/ MCD60_Volante/ TargetPosition
Topics compartidos (relativos)	/<ns>/<Topic>	/ascod/CAN, /ascod/Heartbeat
Salida de tramas CAN	/<ns>/<can> (valor del parámetro can)	/ascod/CAN_7_EPOS

Si no estás seguro del nombre exacto, ejecuta `ros2 topic list` con el sistema arrancado y busca el prefijo de tu vehículo.

4. Topics que PUBLICA el driver (salidas)

En esta sección <ns> = namespace del vehículo, <Nodo> = nombre de ejecución del driver.

4.1 /<ns>/<Nodo>/Status — estado del motor

- **Tipo:** `insia_msg/msg/EPOSStatus`
- **Frecuencia:** ~10 Hz
- **Para qué:** es el topic **más importante para el usuario**. Te dice en qué estado está el motor, en qué modo, su posición y el error de seguimiento.

Campos:

Campo	Tipo	Significado
header	<code>std_msgs/Header</code>	Sello de tiempo
status	<code>string</code>	Estado de la máquina EPOS (ver sección 6). ' ' si aún no se conoce
operation_mode	<code>string</code>	Modo activo: PPM, CST, PVM, HMM, CSP, CSV. ' ' si desconocido
position	<code>int32</code>	Posición actual del motor (unidades del

Campo	Tipo	Significado
		encoder)
following_error	int16	Error de seguimiento actual

```
ros2 topic echo /ascod/MCD60_Volante/Status
```

4.2 /<ns>/Heartbeat — señal de vida

- **Tipo:** insia_msg/msg/StringStamped
- **Frecuencia:** 1 Hz
- **Para qué:** indica que el nodo sigue vivo. El campo data contiene el **nombre del nodo** que emite. Lo usan los supervisores para vigilar que todos los nodos están activos.

Campo	Tipo	Significado
header	std_msgs/Header	Sello de tiempo
data	string	Nombre del nodo (p.ej. MCD60_Volante)

4.3 /<ns>/<can> — tramas CAN salientes (uso interno)

- **Tipo:** insia_msg/msg/CANGroup
- **Para qué:** es la **salida hacia el bus**. El driver empaqueta aquí las tramas CANopen que hay que enviar al motor (inicialización, consignas, estado, etc.). El nombre del topic es el valor del parámetro can (p.ej. CAN_7_EP0S, can_control).
- **No necesitas publicar ni leer este topic manualmente;** lo consume el nodo gateway CAN.

Estructura de CANGroup: header, msg_type, dispositivo y can_frames (lista de mensajes CAN). El driver solo rellena can_frames.

5. Topics a los que se SUSCRIBE el driver (entradas)

Estos son los topics en los que **tú (u otro nodo) publicas** para dar órdenes al motor.

Muy importante:

- TargetPosition solo tiene efecto si el driver está en modo **PPM**.
- TargetTorque solo tiene efecto si el driver está en modo **CST**.

- En ambos casos, el motor debe estar en estado **Operation enabled** (envía primero Enable). Si no, la orden se ignora (queda registrada en el log como *debug*).

5.1 /<ns>/<Nodo>/Enable — habilitar / deshabilitar el motor

- **Tipo:** insia_msg/msg/BoolStamped — campos: header, bool data
- data: true → el driver lleva el motor a **Operation enabled** (listo para moverse).
- data: false → lo lleva a **Switched on** (energizado pero sin movimiento).

5.2 /<ns>/<Nodo>/TargetPosition — consigna de posición (modo PPM)

- **Tipo:** insia_msg/msg/EP0SConsigna — campos: header, int32 position, uint8 mode
- position: posición objetivo (se multiplica internamente por el parámetro factor).
- mode: 0 = **RELATIVO** (incremento sobre la posición actual), 1 = **ABSOLUTO**.

5.3 /<ns>/<Nodo>/TargetTorque — consigna de par (modo CST)

- **Tipo:** insia_msg/msg/IntStamped — campos: header, int32 data
- data: par objetivo a aplicar.

5.4 /<ns>/<Nodo>/Digital — salida digital

- **Tipo:** insia_msg/msg/EP0SDigital — campos: header, uint8 io_digital, bool enable
- io_digital: número de salida digital (válido de 1 hasta el valor del parámetro digital_outputs). Si está fuera de rango, se registra un aviso y no se hace nada.
- enable: true/false para activar/desactivar esa salida (p.ej. relés del volante).

5.5 /<ns>/<Nodo>/Analog — salida analógica

- **Tipo:** insia_msg/msg/EP0SAnalog — campos: header, uint8 io_analog, float64 voltaje
- io_analog: número de salida analógica, válido **1 o 2**.
- voltaje: tensión a aplicar, válida en el rango **-4 a 4 V**. Fuera de rango se ignora con un aviso.

5.6 /<ns>/<Nodo>/FaultReset — resetear un fallo

- **Tipo:** std_msgs/msg/Header (mensaje estándar de ROS2)

- Cualquier mensaje fuerza un **reset de fallo**, independientemente del estado actual.

5.7 /<ns>/<Nodo>/ResetPosition — fijar posición de referencia

- **Tipo:** insia_msg/msg/IntStamped — campos: header, int32 data
- Ajusta la referencia de posición del motor al valor data (operación tipo *homing*).

5.8 /<ns>/CAN — tramas CAN entrantes (uso interno)

- **Tipo:** insia_msg/msg/CAN
 - Lo publica el gateway CAN con cada trama recibida del bus. El driver la **decodifica** según su cobid y su diccionario, y actualiza su estado interno (Statusword, posición, fallos...). **No necesitas publicar aquí manualmente.**
-

6. Máquina de estados y flujo típico

El motor sigue la máquina de estados estándar EPOS/CiA-402. Los valores que verás en `Status.status` son:

Not ready to switch on · Switch on disabled · Ready to switch on · Switched on · Operation enabled · Quick stop active · Fault reaction active · Fault

Flujo típico para mover un motor:

1. **Habilitar:** publica en `.../Enable` con data: `true`.
 2. **Esperar** a que `.../Status` muestre status: `Operation enabled`.
 3. **Mover:** publica la consigna:
 - en modo **PPM** → `.../TargetPosition`
 - en modo **CST** → `.../TargetTorque`
 4. **(Si hay relés/salidas)** activa la salida digital que corresponda con `.../Digital`.
 5. **Ante un fallo** (status: `Fault`): publica en `.../FaultReset`. Nota: si el parámetro `auto_fault_reset` está a `True`, el driver intenta recuperarse solo.
 6. **Al terminar:** publica `Enable` con data: `false` y pon las salidas digitales a `false`.
-

7. Parámetros del driver

Se definen en los ficheros `conf/<vehículo>.yaml`, dentro del bloque del nodo, bajo `ros__parameters`.

Parámetro	Tipo	Obligatorio	Por defecto	Descripción
<code>cobid</code>	<code>int</code>	Sí	—	Dirección (COB-ID) del nodo CANopen del motor
<code>mode</code>	<code>str</code>	Sí	—	Modo de operación: PPM (posición) o CST (par). Decide qué consigna obedece
<code>can</code>	<code>str</code>	Sí	—	Nombre del nodo/topic CAN de salida (debe coincidir con un <code>CAN_Node</code>)
<code>speed</code>	<code>int</code>	Sí	—	Velocidad de perfil (rpm) usada al inicializar el motor
<code>driver_type</code>	<code>str</code>	Sí	—	epos (MCD 60W) o epos4 (EPOS4). Selecciona la

Parámetro	Tipo	Obligatorio	Por defecto	Descripción
				implementación interna
digital_outputs	int	Sí	—	Número de salidas digitales disponibles
auto_fault_reset	bool	Sí	—	Si True, el driver intenta recuperar fallos automáticamente
dictionary	str	Sí	—	Fichero de diccionario de decodificación (epos_dictionary.yaml, epos4_dictionary.yaml)
factor	número	No	1	Multiplicador aplicado a la consigna de posición (position * factor)
log_level	int	No	10	Nivel de log (menor = más detalle)
status_freq	número	No	2	Frecuencia

Parámetro	Tipo	Obligatorio	Por defecto	Descripción
				de sondeo de E/S (uso interno menor)

Notas:

- `driver_type` con un valor distinto de `epos/epos4` provoca un error al arrancar.
- Los diccionarios están en `INSIA_control/diccionarios/` y describen cómo interpretar cada trama CAN recibida.
- El factor del parámetro es **distinto** del factor que aparece dentro de los ficheros de diccionario (ese último escala cada señal decodificada).

8. Valores usados en cada vehículo

Instancias del driver Maxon configuradas en los `.yaml` de `conf/`:

ASCOD — `conf/ascod.yaml`

Nodo	cobid	drive r_type	mode	speed	digit al_ou tputs	auto_ fault_ rese t	facto r	can
MCD60 _Vola nte	3	epos	PPM	2000	4	True	—	CAN_7 _EP0S
EP0S4 _Fren o	1	epos4	PPM	5000	2	True	1	CAN_7 _EP0S

iMiEV — `conf/imievcn.yaml`

Nodo	cobid	drive r_type	mode	speed	digit al_ou tputs	auto_ fault_ rese t	can
EP0S4 _Vola nte	2	epos4	CST	2000	2	False	can_c ontro l

En el iMiEV el **freno** no usa driver Maxon (usa FAULHABER_Freno), por eso no aparece aquí.

Maqueta / GDELS — conf/maqueta.yaml

Nodo	cobid	drive r_typ e	mode	speed	digit al_ou tputs	auto_ fault _rese t	facto r	can
MCD60_Volante	3	epos	PPM	5000	4	True	1	can_control
MCD60_Freno	1	epos	PPM	5000	4	True	1	can_control

9. Ejemplos de uso

Los ejemplos siguientes están tomados de los nodos de control reales del proyecto (INSIA_control/DevicesControlNodes/), que son los que en producción mandan órdenes al driver Maxon. Cada ejemplo muestra **el comando de terminal** (ros2 topic pub) para probarlo a mano y **el patrón en Python** tal y como aparece en el código.

En los ejemplos se usa el vehículo ascod y el motor MCD60_Volante; adapta el namespace y el nombre del nodo a tu caso.

Ejemplo A — Mover el volante en modo PPM (ASCOD)

Referencia: ASCOD/SteeringNode.py. La secuencia es: **habilitar** el motor, **activar el relé** (salida digital 4) y **enviar la consigna de posición** en modo relativo.

Por terminal:

```
# 1) Habilitar el motor
ros2 topic pub --once /ascod/MCD60_Volante/Enable
insia_msg/msg/BoolStamped \
  '{data: true}'

# 2) Activar el relé del volante (salida digital 4)
ros2 topic pub --once /ascod/MCD60_Volante/Digital
insia_msg/msg/EPOSDigital \
  '{io_digital: 4, enable: true}'

# 3) Enviar consigna de posición RELATIVA (mode: 0 = RELATIVO, 1 = ABSOLUTO)
ros2 topic pub --once /ascod/MCD60_Volante/TargetPosition
insia_msg/msg/EPOSConsigna \
  '{position: 5000, mode: 0}'
```

En Python (patrón del SteeringNode):

```
from insia_msg.msg import BoolStamped, EPOSDigital, EPOSConsigna
from std_msgs.msg import Header
from rclpy.qos import HistoryPolicy
```

```
# Publicadores (en __init__)
self.pub_enable = self.create_publisher(BoolStamped,
    'MCD60_Volante/Enable', HistoryPolicy.KEEP_LAST)
self.pub_digital = self.create_publisher(EPOSDigital,
    'MCD60_Volante/Digital', HistoryPolicy.KEEP_LAST)
self.pub_target = self.create_publisher(EPOSConsigna,
    'MCD60_Volante/TargetPosition', HistoryPolicy.KEEP_LAST)
```

```
# 1) Habilitar
self.pub_enable.publish(BoolStamped(
    header=Header(stamp=self.get_clock().now().to_msg()),
    data=True))
```

```
# 2) Activar relé (salida digital 4)
self.pub_digital.publish(EPOSDigital(
    header=Header(stamp=self.get_clock().now().to_msg()),
    enable=True, io_digital=4))
```

```
# 3) Consigna de posición relativa
self.pub_target.publish(EPOSConsigna(
    header=Header(stamp=self.get_clock().now().to_msg()),
    position=5000,
    mode=EPOSConsigna.RELATIVO)) # RELATIVO=0, ABSOLUTO=1
```

Ejemplo B — Controlar el par en modo CST (iMiEV)

Referencia: Imiev/SteeringNode.py. Aquí el motor está en modo CST, así que la consigna es **par** (TargetTorque), no posición.

```
# Habilitar
ros2 topic pub --once /imiev/EPOS4_Volante/Enable
insia_msg/msg/BoolStamped '{data: true}'
```

```
# Consigna de par
ros2 topic pub --once /imiev/EPOS4_Volante/TargetTorque
insia_msg/msg/IntStamped '{data: 150}'
```

```
from insia_msg.msg import BoolStamped, IntStamped
```

```
self.pub_enable = self.create_publisher(BoolStamped,
    'EPOS4_Volante/Enable', HistoryPolicy.KEEP_LAST)
self.pub_target = self.create_publisher(IntStamped,
    'EPOS4_Volante/TargetTorque', HistoryPolicy.KEEP_LAST)
```

```
self.pub_enable.publish(BoolStamped(
    header=Header(stamp=self.get_clock().now().to_msg()), data=True))

self.pub_target.publish(IntStamped(
    header=Header(stamp=self.get_clock().now().to_msg()), data=150))
```

Ejemplo C — Accionar el freno en PPM absoluto (ASCOD)

Referencia: ASCOD/BrakeNode.py. El freno usa consigna de posición **ABSOLUTA** (mode: 1).

```
ros2 topic pub --once /ascod/EPOS4_Freno/Enable
insia_msg/msg/BoolStamped '{data: true}'

# Posición absoluta (p.ej. freno a fondo dentro del rango del
dispositivo)
ros2 topic pub --once /ascod/EPOS4_Freno/TargetPosition
insia_msg/msg/EPOSConsigna \
    '{position: 20000, mode: 1}'

from insia_msg.msg import BoolStamped, EPOSConsigna

self.pub_enable = self.create_publisher(BoolStamped,
'EPOS4_Freno/Enable', HistoryPolicy.KEEP_LAST)
self.pub_target = self.create_publisher(EPOSConsigna,
'EPOS4_Freno/TargetPosition', HistoryPolicy.KEEP_LAST)

self.pub_enable.publish(BoolStamped(
    header=Header(stamp=self.get_clock().now().to_msg()), data=True))

self.pub_target.publish(EPOSConsigna(
    header=Header(stamp=self.get_clock().now().to_msg()),
    position=20000,
    mode=EPOSConsigna.ABSOLUTO)) # ABSOLUTO=1
```

Ejemplo D — Salidas digital y analógica

```
# Activar la salida digital 1
ros2 topic pub --once /ascod/MCD60_Volante/Digital
insia_msg/msg/EPOSDigital \
    '{io_digital: 1, enable: true}'

# Aplicar 2.5 V a la salida analógica 1 (rango válido: -4 a 4 V, io 1
o 2)
ros2 topic pub --once /ascod/MCD60_Volante/Analog
insia_msg/msg/EPOSAnalog \
    '{io_analog: 1, voltaje: 2.5}'
```

Ejemplo E — Resetear un fallo y fijar la posición de referencia

Resetear un fallo (Fault) del motor

```
ros2 topic pub --once /ascod/MCD60_Volante/FaultReset  
std_msgs/msg/Header '{}'
```

Fijar la posición de referencia actual a 0 (homing)

```
ros2 topic pub --once /ascod/MCD60_Volante/ResetPosition  
insia_msg/msg/IntStamped '{data: 0}'
```

```
from std_msgs.msg import Header  
from insia_msg.msg import IntStamped
```

```
self.pub_fault_reset = self.create_publisher(Header,  
'MCD60_Volante/FaultReset', HistoryPolicy.KEEP_LAST)  
self.pub_reset_pos = self.create_publisher(IntStamped,  
'MCD60_Volante/ResetPosition', HistoryPolicy.KEEP_LAST)
```

```
self.pub_fault_reset.publish(Header(stamp=self.get_clock().now().to_ms  
g()))  
self.pub_reset_pos.publish(IntStamped(  
    header=Header(stamp=self.get_clock().now().to_msg()), data=0))
```

10. Diagnóstico rápido

- **Ver el estado del motor:**

```
ros2 topic echo /<ns>/<Nodo>/Status
```

Observa status y operation_mode.

- **El motor no se mueve al enviar una consigna.** Comprueba:
 1. Que el status sea Operation enabled (envía Enable {data: true} antes).
 2. Que el mode del driver coincide con la consigna: PPM → TargetPosition, CST → TargetTorque. Si envías TargetPosition a un driver en CST (o viceversa), la orden **se ignora**.
 3. Que publicas en el topic correcto (ros2 topic list para confirmar namespace y nombre).
- **El motor está en Fault.** Publica en .../FaultReset. Si sigue en fallo, revisa el log del nodo (el mensaje de fallo se registra como aviso, p.ej. *Over Current Error, Following Error, CAN Bus Off...*).

- **No aparecen los topics.** Verifica que el driver está lanzado (revisa el `launch/` del vehículo) y que `vehicle.yaml` define el `id_vehicle` esperado.
 - **No llega estado del motor (`status vacío`).** Suele indicar que no hay comunicación CAN: revisa que el gateway `CAN_Node` correspondiente (parámetro `can`) esté activo y conectado al bus.
-

Documento de referencia del driver `Maxon_Node.py`. Para el detalle de bajo nivel de las tramas `CANopen`, consulta la documentación del fabricante en `doc/Maxon/` y la implementación en `INSIA_control/utils/epos.py/epos4.py`.